# ספרחוב · Seferehov — features, hardware and capacity

Updated 2026-10-21


- What it is
- Features
  - Map and street libraries
  - Adding books — three ways
  - Reading the photos (on the server)
  - The approval queue
  - Reading order (when many readers scan at once)
  - My books (home library)
  - Books for sale or to give away
  - Finding books
  - Reviews, fixes, points
  - App and updates
  - For the admin
  - Measured accuracy (2026-10-21)
- Hardware
- Capacity: how many scans the GPU can handle
  - Ways to grow, in order
- Disk space
- Backups
- Future plans
- Keeping this document current


## What it is

Seferehov (ספרחוב, Hebrew for “street books”) is a web app, installable on any phone, where books pass from hand to hand. It has three parts:

- a shared map of free street libraries (book boxes, little free libraries, shelves in lobbies and parks) and the books on their shelves now;
- readers’ own home libraries, catalogued from photos;
- books readers offer from home, for sale or to give away.

It runs at https://seferehov.com. Hebrew is the default (right to left); English is one tap away. All the AI runs locally on one home server, so there is no per-use AI cost and no photos are sent to an AI company.

## Features

### Map and street libraries

- **Map** of about 48,000 street libraries worldwide, seeded from OpenStreetMap (`amenity=public_bookcase`). About 70 of them are in Israel. When zoomed out, libraries are grouped into number bubbles.
- **Library page**: photo, hint for finding it, the books seen there lately, books taken or gone in the last 90 days, and spines read from photos that no catalogue knew yet (so the next visitor can photograph their covers).
- **Add a library**: GPS position, a photo and a hint. A double tap does not create two libraries.

- **Suggest changes**: new name, hint, photo, position, or “it’s gone”. New libraries and changes wait for a reviewer, and admins get an e-mail digest.
- **Duplicate libraries** within 20 m are listed for merging; old links redirect to the kept one.
- **Printable sign** (A5 with a QR code) for each library.
- **Private-info check**: names, hints and notes are checked for door codes, phone numbers and e-mails (patterns plus the local AI model) before they are published.
- **Town pages and country pages** for search engines, with the libraries and books in each town (towns come from GeoNames and OpenStreetMap boundaries).
- Only libraries with something of their own (added, checked, edited or reported by readers, or in Israel) are in the sitemap and open to search engines; the untouched OpenStreetMap shelves abroad, and towns and countries made only of them, get `noindex` so Google doesn’t treat them as duplicates.

## Adding books — three ways

1. **One camera for everything, free style** (main way, at a library and at home, `web/src/capture.js`): one button, “Photograph books”. The reader points at a shelf, a front cover or a back cover, in any order, and stops for a moment; every stop is a photo. The phone does not decide what a photo is: the server does. (Until 4 Nov 2026 the covers camera decided on the phone — turned over = the same book, moved sideways = the next book — and on a real session of books standing on a shelf it split 3 books into 6: a book turned over on a shelf moves in the picture just as the phone moving on does.)
    - **When a photo is taken.** About 7 times a second the phone lines up a tiny grey copy of the picture with the one before, which gives the phone’s real speed and how far it has moved since the last photo. Once the picture is new (moved on by 40% of it, or changed a lot: a book turned over, the next cover) and the phone is held steady for 0.6 s (0.4 s caught books half-turned), or roughly steady for 1.6 s, the sharpest frame of the pause is kept at full camera resolution. A bar shows “Hold steady…” filling up, then “✓ Taken — turn the book over or move on”.
    - **What goes with each photo.** Its number in the session, the time since the camera opened, a barcode the live camera saw just before (the browser’s barcode reader looks every 0.6 s while the phone is still), the picture’s travel since the last photo and the phone’s own movement from its motion sensors (summed acceleration, turning). The movement is logged for later; the pairing does not use it yet.
    - **Sent one by one.** Each photo is saved on the phone and sent by itself at once (store and forward, below), to `POST /api/captures`. When the camera closes for any reason the server is told how many photos were taken (`POST /api/captures/<session>/end`).
    - **Stopping.** A big “■ Done, send · ” button sits on the camera picture itself (it used to be under the photos, off the screen on a phone), plus ✕ in the corner. After stopping, the panel follows the uploads — “5 of 12 photos sent…”, then “All 12 photos are on the server. You can put the phone away.” — so the reader knows when it is safe to switch the screen off. Switching the screen off with the camera open still works (every photo is already saved on the phone), but the phone may hold the uploads until the app is opened again.
    - **Sorted on the server** (`server/src/capture.js`, `server/src/pairing.js`) as soon as all the photos are there; if the phone never said “end” (the page died), 3 minutes after the last photo; if photos are still missing, 30 minutes after. For each photo the vision model says shelf, front, back or other, and barcodes are read (a cover with a barcode is a back). Covers are cut out; the same side photographed twice is grouped (grey-picture likeness, colours, and the model when unsure). Fronts and backs no more than 5 photos apart are

scored: colour likeness (a book's two covers share paper, colours and publisher's logo), closeness in the session, a back after its front, and the vision model shown both covers ("the same book?" — it says yes too easily, so it only tips close calls). Best pairs first; leftovers are books with one side only. Runs of shelf photos (up to 8) become a shelf scan, each book a covers scan, read as before; everything waits in the approval queue. A book with only its back cover is read from the back (barcode, and the title its blurb names), marked low quality. Photos arriving after their session was sorted are sorted on their own. The sorting is traced (`/admin/trace`, `capture-…`).
    - **Tested** on the real session that went wrong (10 photos: 3 books front and back, fronts photographed twice, one back alone): sorted and paired 10/10 right, about 4 minutes on the home server.
2. **Photos from the gallery**: up to 40 photos at a time, shelves or covers, sorted and paired the same way (only their order is known).
3. **Barcode scanning** (library page): the camera keeps scanning, each ISBN goes to the queue.

**One book's covers on their own** (the covers camera, `web/src/shots.js`): used where a single book is asked for — a missing side on a book page, a spine found in no catalogue (library check), an unknown barcode. The live camera takes each picture by itself when the book is held steady (it accepts more shake the longer it waits, so a photo comes within 1–2 seconds), shows the book turning over between sides, and looks for the barcode the whole time. If none was seen, it asks for a 4-second close-up of the barcode. Then, for 6 seconds, the book's two pictures are shown over the camera with "retake" on each (the first take is sometimes too quick; tapping one goes back to that side) and a bar running down; untouched, or once the next book's front is taken, the book is sent by itself. No "keep" button; a wrong book is declined in the approval queue. - **The whole book must be in the picture.** A small model on the phone (a layer on the same image embedder as the book detector, `web/src/wholehead.json`) says whether all four corners are in view; if not, the photo is held back and the camera says to move back a little, because the server cannot cut out and straighten a cover that is partly off the picture. It was trained on real cover photos of this app, cut so that the cover fits or does not, and on Open Library covers on real surfaces. On real photos it had not seen: 91% of whole covers accepted, 87% of cut ones caught (90% when a quarter or more is missing). After 6 seconds of holding back the photo is taken anyway, so a cover the model misjudges cannot stop the camera; the camera report counts these. A phone that cannot run the model takes photos as before. - **More frames.** Right after each cover photo, while the phone is almost not moving, up to two more frames (1600 px) go with it. No extra time for the reader. For Hebrew covers the server reads each with OCR: lines that are the same text vote, and every frame's spelling of the title is also tried against the catalogue; a frame after the back cover is also searched for the barcode. Tested on a real shelf video: frames differ in which words they get right, while one sharpened picture stacked from the frames read far worse than any single frame, so frames are read, not merged. The shelf camera still sends one frame per pause (its reading takes minutes per photo).

**Store and forward (nothing taken is lost, no button to press).** Every photo, cover, shelf batch, barcode and new library is written to the phone first (IndexedDB) and sent from there by an outbox (`web/src/queue.js`):

- A photo is saved the moment it is taken, into a "held" batch (the shelf camera's photos, a book's two covers) that is sent once complete or as soon as the camera closes for any reason. A batch left by a page that is gone (closed, reloaded, crashed) is sent by itself after 3 minutes.
- No signal, a timeout, the server busy or restarting (5xx, 429): kept and tried again with growing pauses (15 s up to 15 min), at once when the phone is back online or the app is opened. Signed out (401): kept until sign-in. App too old (426): kept and sent by the updated

app (updates no longer wait for the outbox to be empty, only for a send in progress or a batch being taken).

- The service worker sends too (Background Sync): uploads left when the app is closed go when the phone has signal (Chrome/Android).
- Refused by the server (another 4xx): kept as “not sent” with the reason; the header badge says so and lists them, to try again or delete; also retried when the app starts (up to 6 times). A refusal the reader is watching (a mistyped ISBN) is said at once instead.
- A barcode the catalogue knows goes straight to the library (or My books); one it does not know falls back to sending the covers to be read.
- The badge lists what is still on the phone, each with its picture, when it was taken, what went wrong the first time and the latest time, in plain words, and how many tries. “Try again” sends them at once, and the list comes back saying what that try did (how many went, how many are still there).
- A barcode is credited only to the book in view: one read while the picture has not moved on since the last photo belongs to that last book, and a barcode that already went with a book in this session is never given to another.
- A barcode no catalogue knows (usual for a Danacode, the code on most Israeli books, the first time anyone scans that book): the covers go to be read instead, with the barcode, and the book waits in the approval queue.
- Every upload carries an id (`X-Upload-Id`); the server keeps the answer 14 days (`upload_receipts`, `server/src/receipts.js`), so an upload sent again because its answer was lost is never saved twice.

**Camera and upload reports** (for later analysis, to improve scanning; no pictures, numbers only, kept a year): - Every live-camera session — free-style capture, one book’s covers, barcode scanning — sends a report when it ends, also when the camera would not open (`web/src/camlog.js` → `POST /api/camera-log` → table `camera_sessions`). The report goes before anything else is awaited (a page hidden by the screen going off may be frozen at any moment: a covers session’s report was lost this way on 3 Nov 2026), is trimmed to fit the 64 KB a beacon may carry, and is kept on the phone and sent with the next session if it could not go. - The phone: model, system and version (Android Chrome gives them directly; iPhones through the browser’s user-agent text), browser and full version, screen size and pixel ratio, CPU cores, memory, network type and speed, storage used, graphics chip, installed app or browser tab, which camera features the browser has. - The camera actually given: resolution, frame rate, focus, exposure, zoom, torch, what it can do at most, and the largest real photo it can take. - Timings: how long the camera took to open and show a first picture; each photo’s time, how it was taken (shelf: steady / rough / leaving; covers: still / sharpest-of-a-moment / button / camera app), the wait from “ready” to the photo, sharpness, shake, glare or dark, size, time to encode it; each book’s time from front to back and in all, retakes, how its barcode was found (live camera, from the photo, or none) and when; time spent in each camera state; barcode-detector type and time per look; the book detector’s time per look (many books). - Each upload from the outbox (`upload_timings`, `server/src/receipts.js`): how long it waited on the phone (no signal, batch still being taken), how long the upload took on the phone, how long the server took, try number, size, network, page or service worker, and what it became (scan, book or sighting id). The camera session id ties uploads (and so scans and their traces) to the session. - Each line on screen follows its upload: saved on the phone, sending, sent (with the book’s title when known), waiting (no signal / sign in / server busy), or not sent.

## Reading the photos (on the server)

- **Shelf photos**: the local vision model (Qwen3-VL 8B) finds each spine, straightens the photo by the spines' lean, cuts out each spine, and reads it. A second reader (Surya OCR) reads the same cut-out. A reading is kept when the two agree. Readings of the same spine in different photos are merged by similarity, and the most frequent spelling wins.
- **Covers**: the cover is cut out and straightened, then read. Hebrew text is read by Surya OCR and corrected with the hspell dictionary; Latin text by the vision model. Hebrew the model writes back to front is turned round.
- **Hebrew covers, title and author**: the vision model understands the layout (which text is the title, which the author) but invents Hebrew letters it cannot make out; Surya OCR copies the letters exactly but sees only lines. So both read the front, and the model also picks the title and author among the OCR lines. Each field the model read is replaced by the OCR line it resembles; when the two spellings differ, the one with more words the dictionary knows wins. A reading with the author first is turned round.
- **Catalogue match check**: a book found by title (or by a printed Danacode) is accepted only when it fits the covers: every word of its main title on one line of the cover text, a word of a long subtitle somewhere on the covers, and one of its authors among the names on the cover (Hebrew names are compared with romanized ones by their consonants: "Carman" = "קרמן"). Otherwise the book waits with the title as read. A title found inside a longer catalogue title no longer counts as a match.
- **Codes**: barcodes, ISBNs printed on the back (check digit verified) and Danacodes (trusted only when the catalogue record passes the check above).
- **Catalogue lookup**: a local copy of Open Library (38.6 million books, refreshed monthly), then the National Library of Israel, Google Books and Open Library online. Covers and descriptions for book pages come from the National Library of Israel, Simania, Open Library and Amazon.

## The approval queue

- Every book found, from any photo or barcode, waits in one queue per reader (`/queue`), one book at a time with a big picture: approve, later, or wrong. The title and author can be fixed before approving.
- Poor shelf reads stay out of the queue: no catalogue knows them, and the read is low quality, title and author are the same words, no word of a Hebrew title is a word, or it is a misread of a book the catalogue found in the same scan. Nothing is deleted: a button at the end of the queue brings them back. Covers are never left out.
- Duplicates from several photos are merged. Decisions can be undone for 14 days.
- The page tells readers that each correction or "wrong" helps improve reading for everyone. Those scans (photos without location data, trace, and the decision) are kept for good to study reading errors.
- A reader who closed the app gets an e-mail when the results are ready.
- While photos are still being read, the queue page says how many and about how long until they are done.

## Reading order (when many readers scan at once)

- Book covers go first: a cover sent while a long shelf batch is being read is read between two of its photos, within a few minutes.

- Readers take turns: everyone's first shelf scan before anyone's second, so one big home library does not hold up everyone else.
- The expected wait comes from the last 30 days of measured reading times.

## My books (home library)

- Books scanned at home stay private. Shelf photos or covers taken from My books become the reader's own books.
- **"I left it in a library"**: one tap reports the book as added to that library.
- Books of friends, or borrowed from friends, can be kept apart from the reader's own.
- With more than 5 books, the list can be filtered by words, genre and language, and sorted by when added, title, author or year.

## Books for sale or to give away

- Any book in My books can be offered: free or for sale (price in ₪), its condition, the town and a note.
- Only the offered book is public, with the town and the reader's screen name. The address, e-mail and the rest of the home library are never shown. Phone numbers and e-mails in the note are refused.
- A buyer writes through the app. The message is e-mailed to the owner, with the buyer's address to reply to (the buyer is told before sending). Limits: 10 messages an hour, 20 a day.
- Offers appear on the public book page (with schema.org `Offer` data, so search engines can show price and condition) and on `/offers`, filtered by town and by free or for sale.

## Finding books

- **Search** by title or author: within 1, 5, 20 or 50 km of the reader or of a town, or anywhere. With no words it lists all books in the area ("Browse all books" lists every book on a shelf).
- **Filters and sorting** on the results: genre, language, period published (before 1950, 1950–79, 1980–99, 2000s, 2010s, 2020 on) and when last scanned (week, month, year). Each choice shows how many of the books found it would leave. Sort by best match, scanned lately, nearest, most copies, newest or oldest, or title. Each book shows its genre, language and year, on how many shelves it is now, how many times readers scanned it, and when it was last seen.
- **Genres** (23, from novel and poetry to Judaism, cooking and reference) are seldom given by catalogues, and never for most Hebrew books, so the local model picks one from the title, authors, publisher and description, in the background when the GPU is idle (about a second a book). A missing language is taken from the script of the title, or from common English words, else guessed by the model. Readers can correct both in "Fix a mistake", and the book page links them to the matching search.
- **Alerts** (watches) for a book, an author or words, near a place or anywhere, by e-mail or push notification.
- **Public book pages** (`/b/<id>`): covers, cover text, ISBN and Danacode, where the book is now, offers, reviews, and links to Open Library, Google Books, WorldCat and Simania.
- **Reader pages** (`/r/<id>`): what a reader read and recommends. Home libraries are not shown.

## Reviews, fixes, points

- **Reviews**: “I read it”, stars, and four guided questions. Copied text and contact details are refused; admins can hide a review.
- **Book fixes**: anyone can fix a book’s title, author, description and so on. Trusted readers’ fixes go live at once; others are voted on. Every change has a history and can be undone.
- **Points and levels** for approved reports, with a daily cap. New readers’ reports are reviewed first; after a few good ones they publish directly. Reports made far from the library always go to review.

## App and updates

- Installable from the browser (PWA), works offline, Hebrew and English.
- **Comic** at `/comic` (“How it works” in the ☰ menu, linked from About): nine drawn panels explaining why Seferehov exists (gems on street shelves are found only by chance) and how it works (photograph a shelf, approve the books, search by title and distance, alerts, take and leave books). Bubbles and captions are page text in both languages (each caption a `<figcaption>` of its picture, every picture with a Hebrew or English description), also rendered by the server for search engines: ComicStory structured data listing the nine pictures with their descriptions, a 1200×630 share picture with its description, and the pictures listed in the sitemap. Newcomers see a one-line pointer to it on the map until they open it or close the line. The drawings were made with FLUX schnell on the home server; they load only when the comic is opened.
- Updates install in the background and switch in between screens, never during an upload or on the camera, queue or review screens. No version numbers to bump.

## For the admin

- `/admin`: activity, readers, search pages, cover texts waiting for approval, reviews, statistics.
- `/admin#camera`: every live-camera session (shelf, covers, many books, barcode; also ones that sent no photo or whose camera would not open): phone (model · system · browser), camera resolution, time to open, photos and how each was taken, median time from ready to photo, books and front→back time, barcodes found, share of time holding, moving on, too fast, dark and glare, median speed, how it ended, and the full numbers; below, the last 100 uploads with their times on the phone, upload and server.
- `/admin/trace`: every AI stage of every scan, step by step: the video and the frames chosen, each cut-out spine, what each model was asked and answered, and why each book was kept or dropped. Test uploads run the full pipeline without saving anything.
- `server/scripts/eval.js`: a fixed test set for measuring reading accuracy (see below).
- Search statistics: which catalogue answered each search and whether the reader kept the match.

## Measured accuracy (2026-10-21)

On 60 synthetic Hebrew back covers with known text, word accuracy was:

| Reader | Clean | Like a phone photo | Hard (tilted, blurred, noisy) |
|---|---|---|---|
| Surya OCR | 98% | 97% | 98% |
| Surya + hspell | 97% | 96% | 97% |
| Vision model | 92% | 76% | 77% |
| Tesseract | 72% | 73% | 28% |

On real photos, the title was found in:

- Hebrew spines: 71% of spines by the vision model alone, 63% by the full pipeline;
- Hebrew covers: 63% by the vision model, 49% by Surya;
- Latin covers: 93–100%.

Real Hebrew covers are harder than synthetic ones, so a word-accuracy figure for them needs a hand-labelled set.

## Hardware

Everything runs on one home server:

| Part | |
|---|---|
| CPU | 48 threads (AMD Threadripper) |
| Memory | 251 GB |
| GPU | NVIDIA GeForce GTX 1080, 8 GB (from 2016) |
| Disks | 1.8 TB NVMe (1.5 TB free) for the app; a separate hard disk for the nightly backup |
| Software | Node 22 + Fastify API, PostgreSQL 16 + PostGIS (Docker), nginx, Ollama (vision model on the GPU), Surya OCR (on the CPU), hspell |

The same server also runs other services. The GPU is shared with other occasional jobs.

## Capacity: how many scans the GPU can handle

The GPU reads one scan at a time; others wait in a queue (covers first, then readers in turn; see *Reading order*). Measured times:

| Scan | Time on the GPU |
|---|---|
| One book’s front and back covers | about 1 minute |
| One shelf photo | about 3.5–7 minutes (each spine is cut out and read twice) |
| One batch of 8 automatic shelf photos | about 30–55 minutes |
| A short video (no longer offered) | about 35–50 minutes |

So in 24 hours of non-stop work the GPU can read about **1,400 books by their covers**, or about **200–400 shelf photos**. That is roughly 2,500–5,000 books from shelf photos (at 10–15 books per

photo).

A home library of 300 books, photographed shelf by shelf, is about 25 photos, which is 2–3 hours of GPU time. In practice:

- one or two readers scanning at the same time: results within minutes to an hour;
- five readers each scanning a home library at the same time: the last one waits 10 hours or more;
- a reader with one book's covers waits only for the shelf photo being read now (a few minutes), not the whole batch.

The bottleneck is the GPU, not the CPU, memory or disk.

### Ways to grow, in order

1. **Smarter queue (code only, free). Done.** Covers before shelf batches, readers in turn, and an estimated wait shown in the app. This removes the worst waits but not the total: five home libraries at once still take 10+ hours in all.
2. **Faster pipeline (code only).** Each spine is read four times now (two orientations, two readers); reading the likely orientation first and skipping the second read when both agree would cut the time considerably.
3. **A better GPU at home.** A used RTX 3090 (24 GB) costs about $700–900 and is roughly 3 times faster for this kind of model (its memory is about 3 times faster). The power supply has to be checked (the card draws up to 350 W). This keeps the "no per-use cost" property.
4. **A rented GPU for overflow.** The worker takes jobs from the database, and the model address is a setting, so a second worker on a rented GPU server can take jobs only when the queue is long. Approximate on-demand prices (October 2026, check before buying):

| Provider | GPU | Per hour | 24/7 for a month |
|---|---|---|---|
| AWS EC2 g4dn.xlarge | T4, 16 GB (about as fast as the GTX 1080) | $0.53 | $384 |
| AWS EC2 g5.xlarge | A10G, 24 GB | $1.01 | $734 |
| OVHcloud | L4, 24 GB | about $1.00 | about $730 |
| RunPod community cloud | RTX 3090 / 4090 | $0.19–0.39 | $140–280 |

Run only when needed (for example 2 hours a day), a rented GPU costs tens of dollars a month. Running one all the time costs hundreds, more than buying a faster card. A rented server is still a machine the operator controls: photos go to it, not to an AI company, and they are not used to train anyone's model.

## Disk space

Measured per item:

| Kept | Size | How long |
|---|---|---|
| Book cover photos (front + back, re-encoded) | about 0.6 MB per book | permanent |
| Shelf photo for the queue + spine cut-outs | about 0.3 MB per photo + 40 KB per book | permanent |
| Full-size copy of each photo as sent and as read, and the video as sent, without location data or sound (for reading again with a better model) | about 3 MB per photo, 20–100 MB per video | 1 year |
| Scans where a reader declined or corrected a reading: those copies, the trace and what the reader decided (to find where reading goes wrong) | as above | permanent |
| Trace of each scan (admin view) | 0.3 MB per cover, 1.6 MB per shelf photo, 25 MB per video | 1 year |
| Database: Open Library catalogue | 14 GB | rebuilt monthly from the public dump |
| Database: readers’ data | a few MB so far | permanent |

The 1-year files are capped by the GPU: at most about 400 photos a day can be read, so they stay under about 700 GB, and far less at real use. Permanent growth is small. For example, 1,000 home libraries of 300 books photographed by shelf come to about 20 GB, and 100,000 books scanned by their covers to about 60 GB. With 1.5 TB free, disk space is not a concern for years.

## Backups

**Now:** every night at 04:00 the server copies the app files to a second disk in the same machine and dumps the database (the last 7 dumps are kept). Files that were replaced or deleted are kept for 60 days. A failed backup sends an e-mail.

This protects against a failed disk, but not against anything that hits the whole machine or house: theft, fire, a power surge, or a mistake or attack that reaches both disks.

**Recommended:**

- **Dump the readers’ data without the Open Library catalogue.** The catalogue can be downloaded again, and without it the nightly dump shrinks from gigabytes to megabytes.
- **A second machine on the home network: a Raspberry Pi with a USB disk works well.** A Pi 4 or 5 with a 1–2 TB USB SSD or hard disk is enough. Use the wired network if possible; Wi-Fi is fine too, because the nightly change is small (megabytes).
  - Let the Pi **pull** the backup over SSH with a read-only key, so a compromised server cannot delete the backups.
  - Keep dated snapshots (for example with borg or restic), not a single mirror.
- **One copy outside the house**, encrypted (borg or restic to a rented server or cloud storage). The important data is under a few GB, so this costs cents to a few dollars a month.
- **Test a restore** once in a while.

# Future plans

**Waiting and speed**

- Faster shelf reading (read the likely orientation first; skip the second reader when the first is sure).
- Overflow worker on a rented GPU when the queue is long; or a faster GPU at home.

**Home libraries and offers**

- Alerts to readers watching a book when someone offers it.
- Offered books in search results.
- A “wanted” list: readers post books they are looking for, matched against offers and (with the owner’s consent) home libraries.
- Marking an offered book as sold or given, with a thank-you to the other reader.

**Second-hand bookshops**

- Shop mode: photograph the shelves to get an inventory that can be searched and priced, with no typing.
- Photograph again later: sold books drop off the list.
- Export of the stock to the shop’s own website or to classified-ad sites; a page for shops with measured numbers from a pilot.

**Accuracy**

- “Already known”: while a cover or spine is in view, the phone sends a small picture fingerprint (the image embedder it already has); if the book is known with both covers, it says so and the back is not needed. Spines seen before recognised without reading them.
- Use the phone’s motion (logged with every free-style photo) in the pairing, once there is data on what turning a book over and moving on look like.
- Harder synthetic test covers (large decorative titles, text over pictures) and a small hand-labelled set, so accuracy on real covers can be measured and published.
- A public “try it” page that shows each AI stage on an uploaded image.

**Operations**

- Off-machine and off-site backups (above).

# Keeping this document current

This file (`docs/FEATURES.md`) is updated with every feature change. The PDF is built from it with `docs/build-pdf.sh`.